

Documento institucional de trabajo académico

21 de abril de 2026

Integrantes: José Alberto Pedraza Serrano • Cristian Gael Díaz Vázquez • Alfredo Manuel Pérez Bohórquez

Profesor: Miguel Ángel Aguilar Cruz

Asignatura: Sistemas Inteligentes

Memoria técnica, bitácora y dossier de presentación

Documentación completa del proyecto

UNIVERSIDAD TECNOLÓGICA DE MÉXICO

Escuela de Ingeniería y Sistemas



UNIVERSIDAD TECNOLÓGICA DE MÉXICO

Proyecto integrador de

Sistemas Inteligentes

SIMÓN DICE

ESP32-S3 | MicroPython | OLED | interacción por botones y sonido

Documento técnico integral

Este expediente reúne la evolución del proyecto durante nueve semanas de trabajo. Se documentan la propuesta, la construcción del prototipo, la lógica de la máquina de estados, la base de conocimientos, las pruebas funcionales, la evidencia fotográfica y las decisiones de depuración que permitieron estabilizar la versión final.

Materia

Sistemas Inteligentes

Profesor

Miguel Ángel Aguilar Cruz

Integrantes

Jose Alberto Pedraza Serrano
Cristian Gael Díaz Vázquez
Alfredo Manuel Pérez Bohórquez

Periodo documentado

9 semanas de trabajo

Fecha

18 de abril de 2026

Herramientas

Thonny, Arduino IDE y Visual Studio Code

Versión preparada para entrega, exposición y respaldo en repositorio.

Ficha general del proyecto

Título del proyecto	Sistema inteligente interactivo tipo “Simón Dice” implementado sobre ESP32-S3
Materia	Sistemas Inteligentes
Profesor	Miguel Ángel Aguilar Cruz
Integrantes del equipo	Jose Alberto Pedraza Serrano Cristian Gael Díaz Vázquez Alfredo Manuel Pérez Bohórquez
Herramientas de desarrollo	Thonny, Arduino IDE y Visual Studio Code
Plataforma de ejecución	ESP32-S3 WROOM con MicroPython, pantalla OLED, cuatro botones, cuatro LEDs y sensor de audio
Periodo documentado	Nueve semanas de trabajo, con dos sesiones por semana
Versión documentada	Versión funcional validada y preparada para repositorio

El presente documento se redactó como memoria técnica y bitácora formal del proyecto. La intención fue dejar una evidencia ordenada del proceso real de trabajo: lo que se pensó al inicio, lo que sí funcionó en laboratorio, los errores que aparecieron durante las pruebas y la forma en que se corrigieron hasta obtener una versión estable.

La redacción mantiene un tono técnico y directo. Se evitó presentar el prototipo como un ejercicio aislado; en su lugar se describe como un sistema con percepción, base de conocimientos, reglas de inferencia, control por estados y respuesta multimodal, que es justamente el enfoque que la materia exige.

Contenido

1. 1. Resumen ejecutivo
2. 2. Introducción
3. 3. Planteamiento del problema y justificación
4. 4. Objetivos, alcance y restricciones
5. 5. Requerimientos del sistema
6. 6. Fundamento de sistemas inteligentes aplicado al proyecto
7. 7. Descripción general del prototipo
8. 8. Arquitectura física, arquitectura de software y diagrama de comportamiento
9. 9. Metodología de desarrollo y organización del trabajo
10. 10. Bitácora técnica de nueve semanas
11. 11. Construcción del prototipo y evidencia fotográfica
12. 12. Pruebas funcionales, incidencias y resultados
13. 13. Análisis técnico del software implementado
14. 14. Consideraciones para repositorio y mantenimiento
15. 15. Conclusiones y trabajo futuro
16. 16. Anexos de código fuente y configuración
17. 17. Complemento de requerimientos y trazabilidad
18. 18. Casos de uso y escenarios operativos
19. 19. Diagramas de actividades por caso de uso

Nota: por tratarse de un expediente de trabajo y no solamente de un reporte breve, la parte final incorpora anexos técnicos amplios con tablas, diagramas y fragmentos completos del código relevante. Esto permite que el documento pueda usarse tanto para evaluación como para consulta posterior.

Resumen ejecutivo

El proyecto consistió en desarrollar un prototipo funcional del juego “Simón Dice” utilizando una tarjeta ESP32-S3, cuatro botones físicos, cuatro LEDs de colores, una pantalla OLED y un mecanismo de activación por sonido. Aunque en apariencia se trata de un juego sencillo, la versión final se diseñó con una estructura propia de un sistema inteligente pequeño: una base de conocimientos persistente en JSON, un motor de inferencia para validar la secuencia, una máquina de estados para gobernar el flujo de ejecución y módulos separados para percepción, visualización y retroalimentación.

El resultado fue un sistema capaz de iniciar por botón o por aplauso, generar secuencias crecientes, mostrarlas en pantalla y con LEDs, recibir la respuesta del usuario, validar cada entrada en tiempo real y reiniciar el proceso cuando se detecta un error. Durante el desarrollo fue necesario corregir problemas de rebote, reinicios de estado, lectura inestable del micrófono y detalles de visualización de la pantalla. La documentación registra esas decisiones porque explican por qué la versión final no es una maqueta improvisada, sino un prototipo depurado a partir de pruebas reales.

La arquitectura elegida permite defender el proyecto con argumentos sólidos ante una revisión exigente. No se dejó toda la lógica dentro de un archivo único; por el contrario, se distribuyó en carpetas y módulos con responsabilidades específicas. El archivo principal inicializa recursos, la base de conocimientos conserva parámetros e historial, la máquina de estados administra el ciclo del juego y cada estado se concentra en una fase del comportamiento. Este orden facilitó localizar fallas y corregirlas sin romper el resto del sistema.

Desde el punto de vista académico, el proyecto demuestra integración de conceptos de sistemas inteligentes, programación embebida y diseño modular. Desde el punto de vista práctico, deja un antecedente útil para escalar el prototipo hacia una versión con gabinete, buzzer dedicado, mejores sensores o almacenamiento de estadísticas por usuario. En consecuencia, la propuesta cumple con el propósito de la materia y, al mismo tiempo, ofrece una base suficientemente seria para ser presentada ante un evaluador externo.

Introducción

La construcción de prototipos interactivos obliga a trabajar en dos niveles al mismo tiempo. Por un lado está la parte física: conexiones, distribución de pines, estabilidad de alimentación, calidad del montaje y respuesta de los actuadores. Por el otro está la parte lógica: secuencia de arranque, control del flujo, memoria del estado actual, validación de entradas y respuesta ante eventos esperados e inesperados. El proyecto “Simón Dice” permitió abordar ambas capas de manera concreta dentro del contexto de la materia Sistemas Inteligentes.

Se eligió este caso de estudio porque, detrás de un juego aparentemente elemental, existe una secuencia clara de percepción, decisión y acción. El sistema observa una activación inicial, genera una secuencia, la comunica al usuario, espera una entrada, compara esa entrada con una secuencia patrón almacenada y decide si la ronda continúa o si debe reiniciarse. Dicho de otra forma, el prototipo contiene los elementos mínimos para explicar cómo una máquina representa información, aplica reglas y produce conductas observables.

La versión desarrollada por el equipo se implementó sobre ESP32-S3 y MicroPython, lo que permitió iterar con rapidez durante las semanas de prueba. En laboratorio, esta elección fue útil porque facilitó la depuración serial y la reorganización del código sin tener que recompilar proyectos complejos a cada cambio. También se trabajó con Thonny, Arduino IDE y Visual Studio Code según la tarea de cada momento: pruebas rápidas sobre la placa, revisión de estructura de archivos y ajuste del código antes de cargarlo al dispositivo.

El reporte no pretende exagerar el alcance del prototipo. Se reconoce que la versión presentada sigue montada en protoboard y que aún hay oportunidades claras de mejora en ergonomía y encapsulado. Sin embargo, sí se sostiene que el trabajo alcanzó un nivel técnico suficiente para ser documentado con formalidad, porque hubo planeación, corrección de errores, validación funcional y una intención real de estructurar el software con criterios de mantenibilidad.

Planteamiento del problema y justificación

Uno de los problemas más frecuentes en proyectos académicos de electrónica o programación embebida es que se obtiene un resultado visible, pero no necesariamente un sistema defendible. En muchos casos el prototipo enciende, parpadea o ejecuta una secuencia, pero el equipo no puede explicar con claridad cómo está organizada la lógica, dónde se guarda la información del sistema o qué reglas se aplican para decidir cada transición. Eso vuelve frágil la exposición del proyecto frente a un profesor o un sinodal exigente.

En este contexto, el reto fue construir un prototipo que no se quedara en el nivel demostrativo. El equipo buscó que el juego “Simón Dice” no dependiera de una rutina rígida y lineal, sino de una base de conocimientos que almacenara parámetros, de reglas de inferencia que evaluaran cada paso y de un esquema explícito de estados. Esa decisión se tomó desde el inicio para que el proyecto pudiera sostenerse como una propuesta de sistemas inteligentes, y no solamente como una práctica de encender LEDs.

La justificación académica del proyecto está en la posibilidad de aterrizar conceptos abstractos en un prototipo físico. La máquina de estados, la inferencia basada en comparación contra una secuencia maestra, la persistencia de datos en JSON y la activación híbrida por botón o sonido son recursos que hacen visible la relación entre software y comportamiento del sistema. En otras palabras, el proyecto permite ver cómo una representación interna modifica las acciones de la tarjeta.

La justificación práctica también es válida. El prototipo puede servir como base para sistemas de entrenamiento de memoria, interfaces reactivas o ejercicios de interacción humano-máquina. Aunque el resultado final todavía es escolar, la arquitectura ya deja preparado el terreno para escalar a una versión con más sensores, mejores salidas de audio o incluso variantes de juego. Documentarlo con detalle tiene sentido porque preserva ese aprendizaje para futuras iteraciones.

Objetivos, alcance y restricciones

Objetivo general

Diseñar, construir y documentar un sistema inteligente interactivo tipo “Simón Dice” sobre ESP32-S3, capaz de percibir una activación inicial, generar secuencias crecientes, validar la respuesta del usuario mediante reglas de inferencia y comunicar el resultado mediante salidas visuales y de audio.

Objetivos específicos

- Implementar una base de conocimientos persistente para almacenar parámetros del hardware y del estado de juego.
- Construir una máquina de estados con transiciones claras entre espera, reproducción de secuencia, captura de entrada y manejo de error.
- Integrar pantalla OLED, LEDs, botones y sensor de sonido en un mismo ciclo de operación.
- Documentar el desarrollo por semanas, incluyendo correcciones realizadas durante las pruebas.
- Preparar una versión final organizada para exposición y respaldo en repositorio.

Alcance del proyecto

La versión documentada llega hasta un prototipo funcional en protoboard con interacción local. El sistema genera secuencias, incrementa la dificultad y muestra mensajes en pantalla. También incorpora disparadores seriales de audio y persistencia básica de parámetros. No se incluyó gabinete, interfaz gráfica avanzada en computadora, conexión a red ni almacenamiento de perfiles de usuario.

Restricciones identificadas

Las principales restricciones fueron el tiempo de laboratorio, la disponibilidad de componentes, el hecho de trabajar todavía sobre protoboard y la necesidad de equilibrar estabilidad con rapidez de iteración. Algunas decisiones de implementación privilegiaron una versión robusta y defendible por encima de una versión visualmente terminada.

Requerimientos del sistema

Requerimientos funcionales

- El sistema debe poder iniciar al detectar un botón o un aplauso por encima del umbral configurado.
- Debe generar una secuencia maestra de colores y ampliarla después de cada ronda superada.
- Debe mostrar la secuencia mediante LEDs y mensajes breves en la pantalla OLED.
- Debe aceptar entradas físicas por medio de cuatro botones asignados a cuatro colores.
- Debe comparar cada entrada con la posición correspondiente de la secuencia maestra.
- Debe reiniciar el juego cuando el usuario comete un error y conservar el récord máximo observado.

Requerimientos no funcionales

- La respuesta del prototipo debe ser suficientemente rápida para que el usuario perciba continuidad entre acción y reacción.
- La organización del software debe ser legible y separada por responsabilidades.
- Los parámetros críticos deben conservarse en un archivo JSON fácil de revisar y ajustar.
- El sistema debe poder depurarse mediante salida serial sin depender de herramientas propietarias.

- La documentación debe dejar evidencia del proceso real y no únicamente del resultado final.

Fundamento de sistemas inteligentes aplicado al proyecto

Para sostener que el prototipo pertenece al ámbito de sistemas inteligentes no basta con que exista una secuencia de botones. La parte relevante es que el dispositivo maneja una representación interna del problema y actúa en función de ella. En este caso, la representación central es la secuencia maestra almacenada en la base de conocimientos, junto con variables como el paso actual, la puntuación, el récord máximo y los parámetros de aceleración del juego.

La toma de decisiones se materializa en el motor de inferencia. Cada vez que el usuario presiona un botón, el sistema consulta qué símbolo esperaba en la posición actual de la secuencia y compara ese valor con la entrada recibida. Si coinciden, se deduce que la acción es válida; si no coinciden, se deduce que la ronda terminó con error. Esta forma de trabajo, aunque sencilla, ya corresponde a una lógica de evaluación basada en conocimiento interno y reglas.

Otro rasgo importante es la separación entre percepción, estado interno y salida. El prototipo no responde únicamente por reflejo. Primero interpreta una señal de inicio, luego cambia de estado, después presenta una secuencia, espera la interacción humana y finalmente decide cómo continuar. Esa separación hace posible explicar el sistema como una entidad que observa, procesa y actúa, que es justamente una manera elemental pero correcta de introducir el enfoque de sistemas inteligentes.

Por último, la persistencia en JSON aporta una ventaja didáctica: el conocimiento del sistema no queda oculto en constantes dispersas dentro del programa. Se concentra en una estructura legible donde aparecen el mapa de hardware, las reglas de inferencia y la memoria de trabajo. Eso facilita tanto la explicación ante un evaluador como la modificación futura del comportamiento del juego.

Descripción general del prototipo

El prototipo final está compuesto por una tarjeta ESP32-S3 WROOM, una pantalla OLED I2C, cuatro botones de entrada, cuatro LEDs de salida, resistencias de apoyo y un micrófono analógico para activación por ruido. El usuario puede comenzar una partida mediante un botón o un aplauso. Una vez iniciada la partida, el sistema muestra en pantalla el nivel y reproduce una secuencia de colores que el usuario debe repetir en el orden correcto.

Si cada entrada coincide con la secuencia almacenada, la ronda se considera superada, se incrementa la puntuación y el sistema agrega un nuevo paso a la secuencia. Si aparece una discrepancia en cualquier punto, el prototipo activa la rutina de error, muestra el resultado final y regresa al estado de espera. El comportamiento completo está gobernado por una máquina de estados, lo que permitió evitar transiciones ambiguas o reinicios parciales.

Arquitectura física del sistema

La distribución física del prototipo se organizó alrededor del ESP32-S3 como nodo central de control. A la izquierda se concentran los elementos de entrada, a la derecha los actuadores visuales y en la parte superior la pantalla OLED y el micrófono. Aunque el montaje sigue siendo sobre protoboard, la asignación de roles entre componentes ya se encuentra claramente definida.

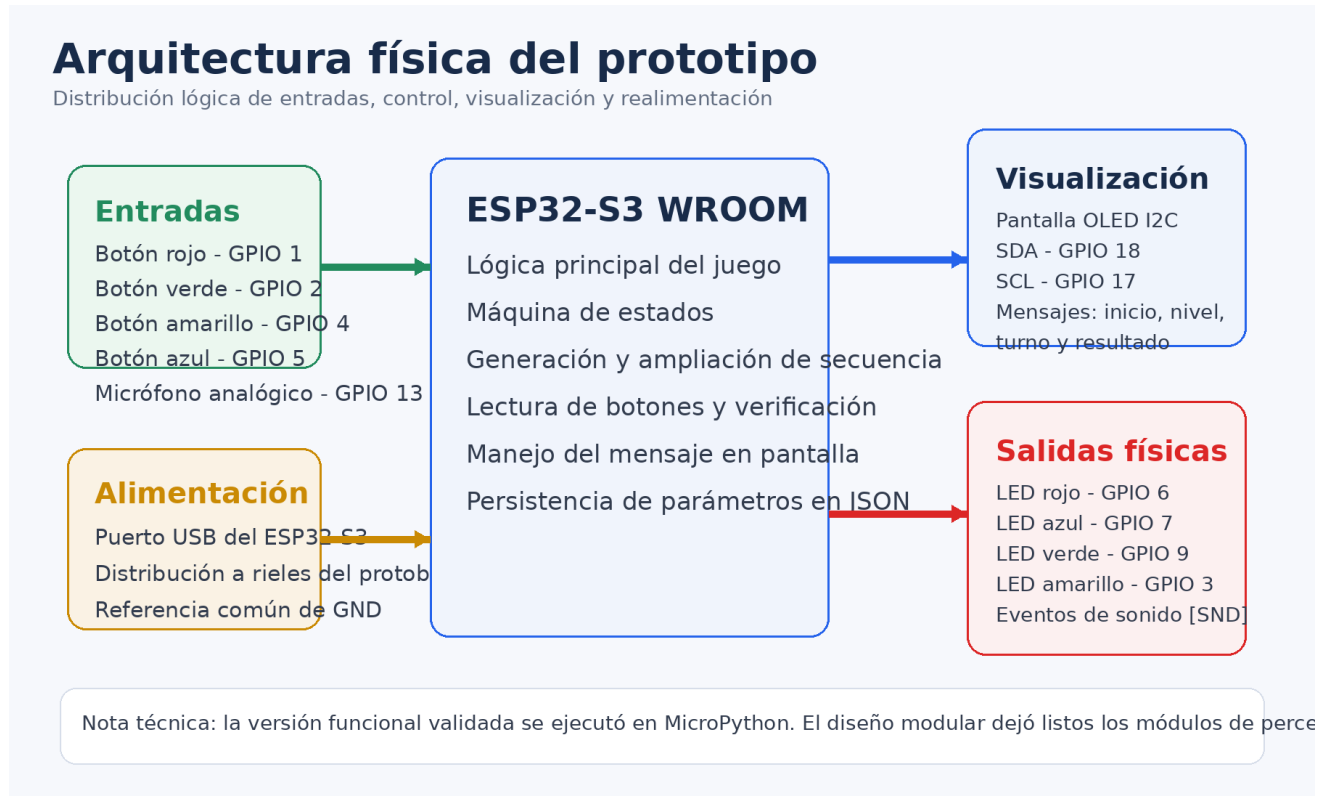


Figura 1. Arquitectura física y distribución lógica de entradas, control y salidas.

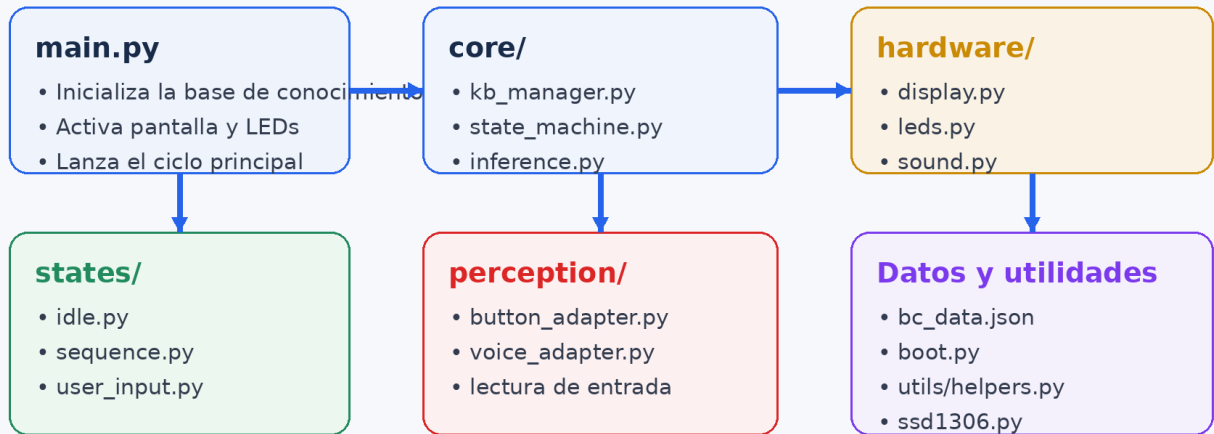
Se buscó que los recorridos de alimentación y referencia a tierra quedaran compartidos por rieles, y que las conexiones críticas de pantalla y botones pudieran seguirse sin dificultad durante la revisión. El montaje todavía puede compactarse, pero la evidencia presentada en laboratorio permitió validar que cada canal funcional quedó correctamente enlazado con el microcontrolador.

Arquitectura de software

La solución de software se diseñó con un criterio modular. La lógica general del juego se encuentra separada por carpetas, y cada archivo cumple una responsabilidad puntual. Este criterio fue importante porque durante el desarrollo aparecieron correcciones sobre la secuencia, la lectura del micrófono y la persistencia del estado; sin esa separación, el mantenimiento se habría vuelto innecesariamente complejo.

Arquitectura de software y organización del proyecto

Estructura modular observada en la carpeta del proyecto y función de cada bloque



La carpeta muestra una evolución hacia una arquitectura separada por responsabilidades. Aunque durante la depuración final algunas lecturas críticas se resolvieron de forma directa dentro de los estados para ganar estabilidad, la base del proyecto quedó organizada para seguir creciendo sin perder claridad.

Figura 2. Organización modular del proyecto y relación entre carpetas.

El archivo `main.py` inicializa la aplicación, `core/` concentra la base de conocimientos, el motor de inferencia y la máquina de estados, `states/` modela cada fase del comportamiento, `hardware/` encapsula pantalla, LEDs y sonido, y `perception/` contiene adaptadores de lectura. La estructura resultante no es decorativa: refleja la forma real en que el sistema se fue estabilizando.

Diagrama de comportamiento del sistema

A solicitud de la revisión final, se integró un diagrama en formato de flujo tipo UML a partir de la lógica descrita por el equipo. El esquema resume la secuencia operativa desde la carga de la base de conocimientos hasta la validación de entradas del usuario y el manejo del error.

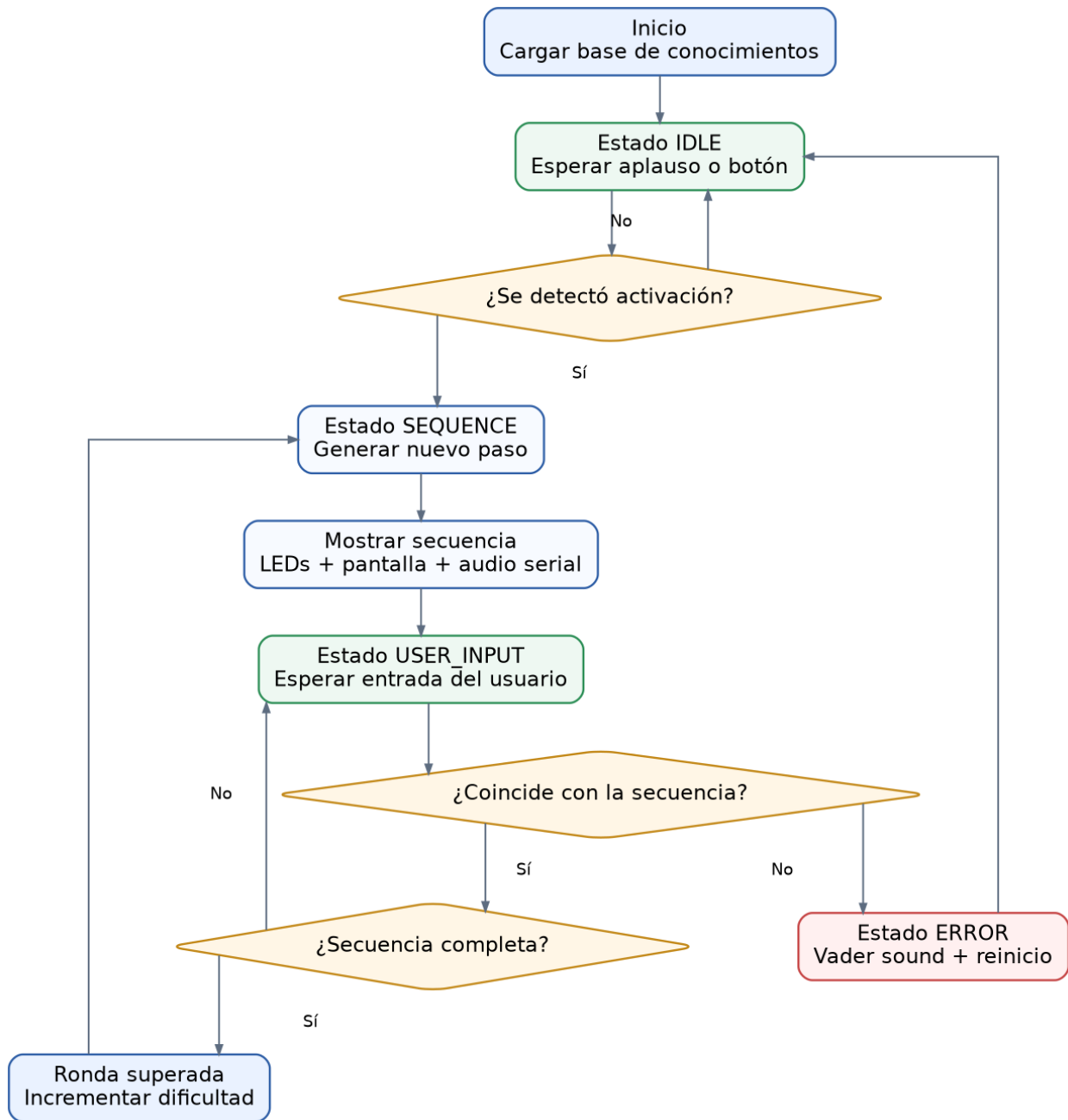


Figura 3. Diagrama de comportamiento derivado del flujo del juego.

El diagrama deja claro que el sistema no permanece en un único bucle monolítico. Cambia entre espera, generación de secuencia, captura de entrada y rutina de error. Esa explicitud es útil tanto para la explicación académica como para la depuración del código.

Organización del repositorio y archivos clave

Estructura de archivos del proyecto

Organización modular utilizada para mantener el código claro y escalable

```
proyecto_simon_dice/  
├── bc_data.json  
├── boot.py  
├── main.py  
├── ssd1306.py  
├── core/  
│   ├── inference.py  
│   ├── kb_manager.py  
│   └── state_machine.py  
├── hardware/  
│   ├── display.py  
│   ├── leds.py  
│   └── sound.py  
├── perception/  
│   ├── button_adapter.py  
│   └── voice_adapter.py  
├── states/  
└── idle.py
```

Figura 4. Árbol general de carpetas y archivos relevantes del proyecto.

La estructuración del repositorio se presenta porque también forma parte del valor del trabajo. Un proyecto puede funcionar en laboratorio y aun así estar mal preparado para entregarse, mantenerse o revisarse. En este caso se buscó que cualquier integrante del equipo pudiera localizar con rapidez dónde se inicia el sistema, dónde se guarda el conocimiento y en qué archivo se encuentra cada estado del juego.

Metodología de desarrollo y organización del trabajo

El proceso de trabajo no se llevó como una cascada rígida, sino como una secuencia corta de diseño, construcción, prueba y ajuste. En la práctica, cada semana se avanzó en dos tipos de actividades: una sesión enfocada en hardware o integración física y otra orientada a lógica de control, depuración o documentación. Este esquema fue conveniente porque el prototipo evolucionó a partir de fallas observadas directamente durante las pruebas.

La metodología empleada puede describirse como iterativa con validación incremental. Primero se montó una versión mínima del circuito, luego se probó el encendido y la lectura de entradas, después se implementó el flujo base del juego, más adelante se estabilizó la pantalla y finalmente se refinaron la base de conocimientos, la inferencia y la organización modular del código. Documentar por semanas ayuda a demostrar que sí hubo un proceso real detrás del resultado final.

Bitácora resumida de desarrollo por semanas

Reconstrucción técnica del proceso seguido a partir del prototipo, el código y la evidencia de pruebas



Figura 5. Línea de tiempo de nueve semanas con los hitos más relevantes.

En términos de coordinación, los integrantes del equipo se apoyaron mutuamente entre revisión del montaje, organización del software, prueba serial y preparación del material para entrega. La división no fue completamente rígida, pero sí existió una tendencia a repartir tareas por afinidad técnica, lo que ayudó a sostener el avance dentro del tiempo disponible.

Materiales, componentes y herramientas

Componente	Cantidad	Función principal	Estado en prototipo	Observación
ESP32-S3 WROOM	1	Control central del sistema	Instalado	Ejecuta la lógica principal en MicroPython
Pantalla OLED I2C	1	Retroalimentación visual	Instalada	Muestra mensajes de estado y color
Botones pulsadores	4	Entrada del usuario	Instalados	Asignados a cuatro colores
LEDs	4	Salida visual	Instalados	Representan la secuencia y el feedback
Resistencias	Varias	Polarización y limitación de corriente	Instaladas	Distribuidas según necesidad del montaje
Micrófono/sensor de audio	1	Activación por sonido	Instalado	Usado para detectar aplauso o ruido
Protoboard y jumpers	1 set	Interconexión y pruebas	Instalados	Permiten reconfiguración rápida
Thonny	1 entorno	Carga y prueba directa en MicroPython	Usado	Facilitó depuración sobre la placa
Arduino IDE	1 entorno	Apoyo de revisión y compatibilidad	Usado	Se utilizó como herramienta complementaria
Visual Studio Code	1 entorno	Edición y organización del código	Usado	Útil para la revisión modular del proyecto

Tabla 1. Materiales, componentes y herramientas empleadas.

Además del componente físico, se considera herramienta a cada entorno que intervino en la preparación del sistema. Esta precisión es importante porque la estabilidad final del prototipo dependió tanto del montaje como de la forma en que se revisó y ordenó el software.

Mapa de pines y variables relevantes

ID lógico	Color	GPIO LED	GPIO botón	Observación
0	rojo	6	1	Asignación tomada de la base de conocimientos
1	azul	7	5	Asignación tomada de la base de conocimientos
2	verde	9	2	Asignación tomada de la base de conocimientos
3	amarillo	3	4	Asignación tomada de la base de conocimientos

Tabla 2. Asignación lógica y física de colores, LEDs y botones.

El hecho de que el mapa se encuentre también en JSON simplifica la exposición del proyecto. Si se cambiara una conexión, la documentación no tendría que esconder el ajuste: bastaría con actualizar la asignación y verificar la congruencia entre montaje y software.

Bitácora técnica de nueve semanas

La siguiente bitácora resume el avance por semana y por sesión. No se presenta como una lista artificial de actividades perfectas; por el contrario, se incluyen correcciones, ajustes y observaciones que realmente formaron parte del proceso. Ese enfoque resulta más creíble y, además, ayuda a defender la evolución del prototipo ante preguntas sobre cómo se llegó a la versión final.

Semana 1: Definición del alcance y primera propuesta de arquitectura

Sesión 1

Se acordó utilizar un juego tipo Simón Dice como problema base, pero desde la primera discusión se decidió que debía justificarse como sistema inteligente. Se listaron los elementos mínimos: secuencia patrón, memoria del progreso, decisión por acierto/error y respuesta física visible.

Sesión 2

Se revisaron componentes disponibles, se eligió ESP32-S3 como tarjeta principal y se trazó una primera distribución en protoboard. También se evaluó qué herramientas de software convenía usar para avanzar rápido durante el laboratorio.

Entregable o avance de la semana	Se obtuvo un incremento comprobable en la funcionalidad o en la organización del proyecto.
Aprendizaje técnico principal	Cada ajuste reforzó la idea de que un sistema estable requiere separar la lógica, limpiar estados y documentar cada corrección importante.

Comentario de cierre de la semana 1: el avance de este periodo fue relevante porque permitió enlazar la parte física con decisiones de software que después se conservaron en la versión final. La continuidad entre una semana y otra no fue artificial; cada ajuste respondió a observaciones concretas obtenidas en laboratorio.

Semana 2: Montaje inicial de entradas y salidas

Sesión 1

Se cablearon los primeros botones y LEDs. En esta etapa el objetivo no fue la estética del montaje, sino comprobar la lectura básica de pines y la activación independiente de cada salida.

Sesión 2

Se realizaron pruebas de continuidad y se detectaron acomodos mejorables en los rieles de alimentación. Se ajustó el tendido de jumpers para evitar confusiones al seguir la lógica del circuito.

Entregable o avance de la semana	Se obtuvo un incremento comprobable en la funcionalidad o en la organización del proyecto.
Aprendizaje técnico principal	Cada ajuste reforzó la idea de que un sistema estable requiere separar la lógica, limpiar estados y documentar cada corrección importante.

Comentario de cierre de la semana 2: el avance de este periodo fue relevante porque permitió enlazar la parte física con decisiones de software que después se conservaron en la versión final. La continuidad entre una semana y otra no fue artificial; cada ajuste respondió a observaciones concretas obtenidas en laboratorio.

Semana 3: Integración de la pantalla OLED

Sesión 1

Se incorporó la pantalla para mostrar mensajes simples de inicio y prueba. La meta fue validar el canal I2C y comprobar que el ESP32-S3 pudiera actualizar texto sin afectar el resto del montaje.

Sesión 2

Aparecieron inconsistencias ocasionales de visualización. Esto llevó a replantear la inicialización del display y a considerar una rutina de rescate en caso de error de comunicación.

Entregable o avance de la semana	Se obtuvo un incremento comprobable en la funcionalidad o en la organización del proyecto.
Aprendizaje técnico principal	Cada ajuste reforzó la idea de que un sistema estable requiere separar la lógica, limpiar estados y documentar cada corrección importante.

Comentario de cierre de la semana 3: el avance de este periodo fue relevante porque permitió enlazar la parte física con decisiones de software que después se conservaron en la versión final. La continuidad entre una semana y otra no fue artificial; cada ajuste respondió a observaciones concretas obtenidas en laboratorio.

Semana 4: Primera versión del flujo del juego

Sesión 1

Se programó una secuencia básica de colores y se definió la estructura general de estados. En esta fase todavía no existía la organización modular final, pero ya se veía la necesidad de separar responsabilidades.

Sesión 2

Se probó la interacción del usuario con una primera lógica de aciertos. Las pruebas mostraron que era indispensable controlar mejor el rebote y el reinicio de variables entre partidas.

Entregable o avance de la semana	Se obtuvo un incremento comprobable en la funcionalidad o en la organización del proyecto.
Aprendizaje técnico principal	Cada ajuste reforzó la idea de que un sistema estable requiere separar la lógica, limpiar estados y documentar cada corrección importante.

Comentario de cierre de la semana 4: el avance de este periodo fue relevante porque permitió enlazar la parte física con decisiones de software que después se conservaron en la versión final. La continuidad entre una semana y otra no fue artificial; cada ajuste respondió a observaciones concretas obtenidas en laboratorio.

Semana 5: Diseño de la base de conocimientos

Sesión 1

Se trasladaron parámetros del sistema a un archivo JSON: mapa de hardware, reglas de tiempo, récord y memoria del estado actual. Esta decisión permitió dejar de depender de constantes aisladas dentro del programa.

Sesión 2

Se implementaron funciones de lectura y escritura para conservar cambios en la base de conocimientos. A partir de aquí el proyecto empezó a adquirir una forma más consistente para la materia.

Entregable o avance de la semana	Se obtuvo un incremento comprobable en la funcionalidad o en la organización del proyecto.
Aprendizaje técnico principal	Cada ajuste reforzó la idea de que un sistema estable requiere separar la lógica, limpiar estados y documentar cada corrección importante.

Comentario de cierre de la semana 5: el avance de este periodo fue relevante porque permitió enlazar la parte física con decisiones de software que después se conservaron en la versión final. La continuidad entre una semana y otra no fue artificial; cada ajuste respondió a observaciones concretas obtenidas en laboratorio.

Semana 6: Motor de inferencia y validación de entradas

Sesión 1

Se formalizó la comparación entre la entrada recibida y la secuencia maestra. La validación dejó de estar dispersa y pasó a concentrarse en un módulo específico.

Sesión 2

Se añadió la lógica para distinguir entre avance parcial, ronda completada y error del usuario. También se comenzó a registrar el récord máximo como evidencia de funcionamiento acumulado.

Entregable o avance de la semana	Se obtuvo un incremento comprobable en la funcionalidad o en la organización del proyecto.
Aprendizaje técnico principal	Cada ajuste reforzó la idea de que un sistema estable requiere separar la lógica, limpiar estados y documentar cada corrección importante.

Comentario de cierre de la semana 6: el avance de este periodo fue relevante porque permitió enlazar la parte física con decisiones de software que después se conservaron en la versión final. La continuidad entre una semana y otra no fue artificial; cada ajuste respondió a observaciones concretas obtenidas en laboratorio.

Semana 7: Activación híbrida y depuración fina

Sesión 1

Se integró la activación por sonido usando un umbral de lectura del micrófono. El objetivo era que el proyecto tuviera un componente perceptivo adicional y no dependiera solamente de botones.

Sesión 2

Las pruebas revelaron ruido e inestabilidad en algunas lecturas. Se ajustó el umbral y se limpió el estado al inicio de una nueva partida para evitar comportamientos erráticos o arranques con datos residuales.

Entregable o avance de la semana	Se obtuvo un incremento comprobable en la funcionalidad o en la organización del proyecto.
Aprendizaje técnico principal	Cada ajuste reforzó la idea de que un sistema estable requiere separar la lógica, limpiar estados y documentar cada corrección importante.

Comentario de cierre de la semana 7: el avance de este periodo fue relevante porque permitió enlazar la parte física con decisiones de software que después se conservaron en la versión final. La continuidad entre una semana y otra no fue artificial; cada ajuste respondió a observaciones concretas obtenidas en laboratorio.

Semana 8: Modularización y evidencia de funcionamiento

Sesión 1

Se reorganizaron carpetas y archivos para dejar una estructura más clara: core, hardware, perception, states y utilidades. Esta etapa estuvo muy orientada a que el proyecto pudiera explicarse con orden.

Sesión 2

Se grabó evidencia de video y se capturaron fotografías de distintas etapas del montaje. También se revisaron mensajes seriales y se añadieron marcas como [SND] para sincronizar eventos de salida.

Entregable o avance de la semana	Se obtuvo un incremento comprobable en la funcionalidad o en la organización del proyecto.
Aprendizaje técnico principal	Cada ajuste reforzó la idea de que un sistema estable requiere separar la lógica, limpiar estados y documentar cada corrección importante.

Comentario de cierre de la semana 8: el avance de este periodo fue relevante porque permitió enlazar la parte física con decisiones de software que después se conservaron en la versión final. La continuidad entre una semana y otra no fue artificial; cada ajuste respondió a observaciones concretas obtenidas en laboratorio.

Semana 9: Cierre técnico y preparación de entrega

Sesión 1

Se revisó la estabilidad de la versión final, se validó la secuencia completa de juego y se preparó el material de defensa. En esta semana se dio especial atención a la presentación del documento y del repositorio.

Sesión 2

Se depuraron detalles de redacción, se consolidaron tablas, diagramas y anexos de código, y se dejó una versión final lista para exposición, revisión académica y respaldo en GitHub.

Entregable o avance de la semana	Se obtuvo un incremento comprobable en la funcionalidad o en la organización del proyecto.
Aprendizaje técnico principal	Cada ajuste reforzó la idea de que un sistema estable requiere separar la lógica, limpiar estados y documentar cada corrección importante.

Comentario de cierre de la semana 9: el avance de este periodo fue relevante porque permitió enlazar la parte física con decisiones de software que después se conservaron en la versión final. La continuidad entre una semana y otra no fue artificial; cada ajuste respondió a observaciones concretas obtenidas en laboratorio.

Construcción del prototipo y evidencia fotográfica

La documentación visual se incorporó no como adorno, sino como evidencia de que el montaje fue cambiando conforme avanzaron las pruebas. En las imágenes se aprecia el acomodo del ESP32-S3, la pantalla OLED, los botones, los LEDs y el tendido de cables. También se observa que la solución fue depurándose sin perder la lógica original del circuito.

En lugar de ocultar que el proyecto sigue sobre protoboard, la estrategia del reporte es reconocerlo y explicar por qué aun así la evidencia es válida. El objetivo de esta entrega es demostrar funcionamiento, estructura y criterio técnico. Una fase posterior podría centrarse en el encapsulado o la fabricación de una tarjeta más compacta.

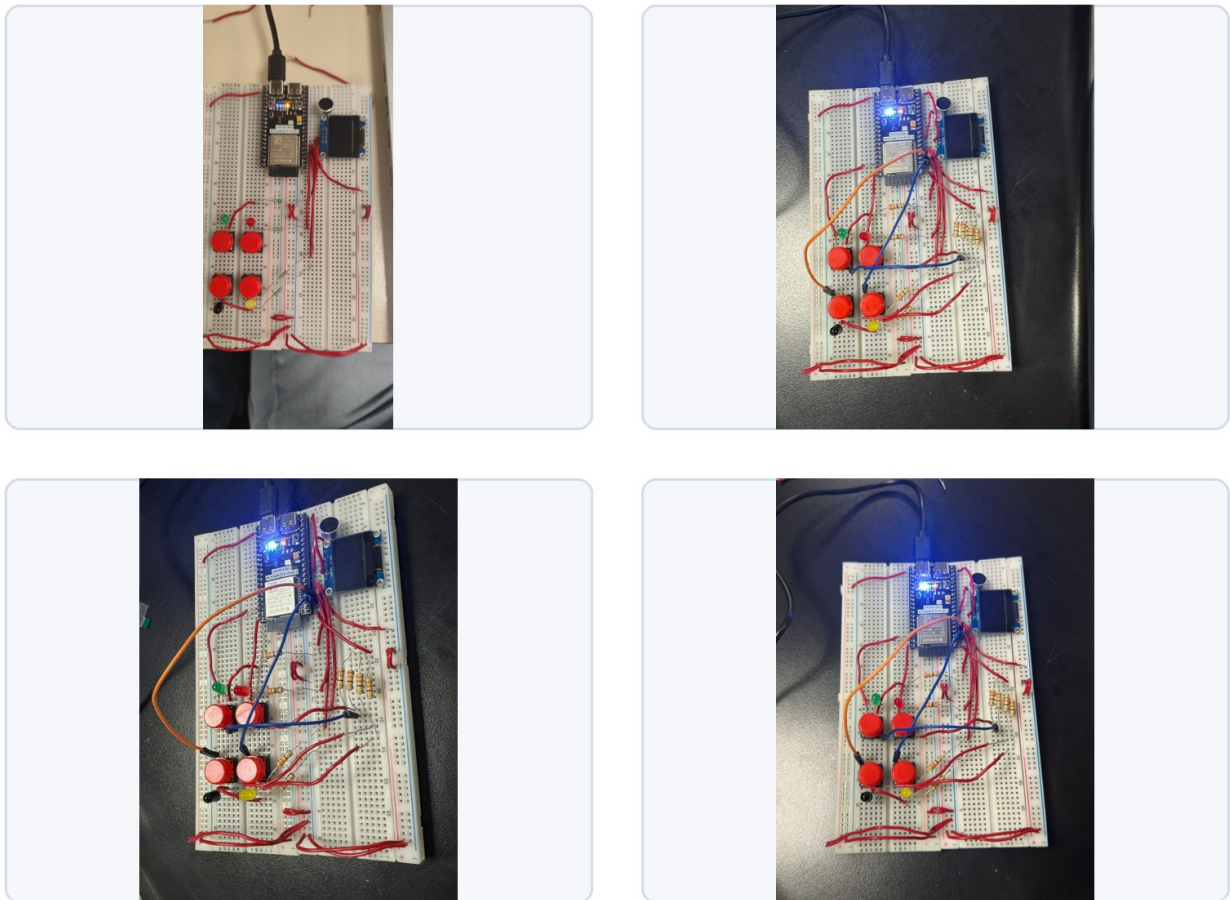


Figura 6. Collage de distintas etapas y ángulos del prototipo.

Evidencia fotográfica y audiovisual del desarrollo

Registro del montaje físico, evolución del prototipo y capturas del video de funcionamiento

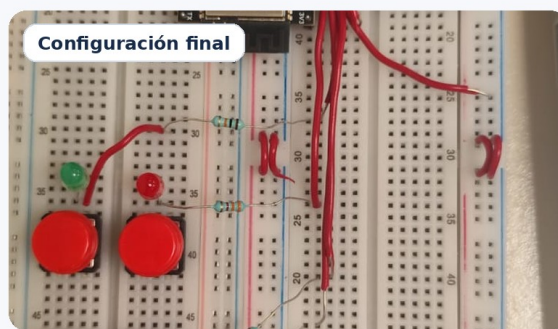
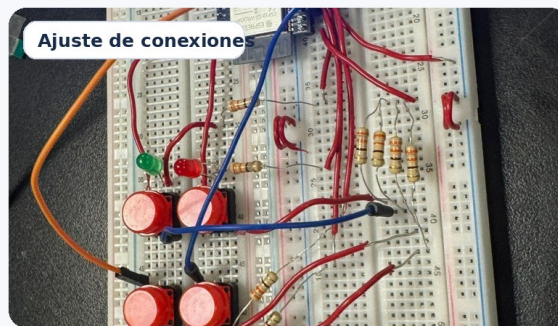
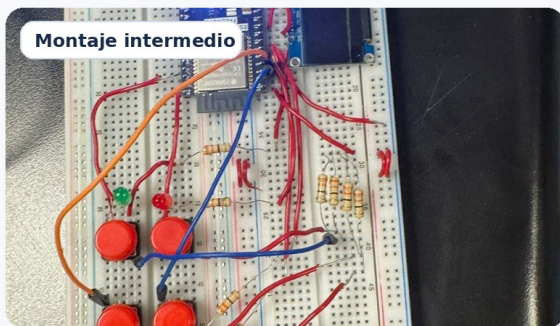


Figura 7. Evidencia consolidada del montaje, cableado y configuración.

Capturas del video de funcionamiento

Secuencia de evidencia tomada del video entregado por el equipo

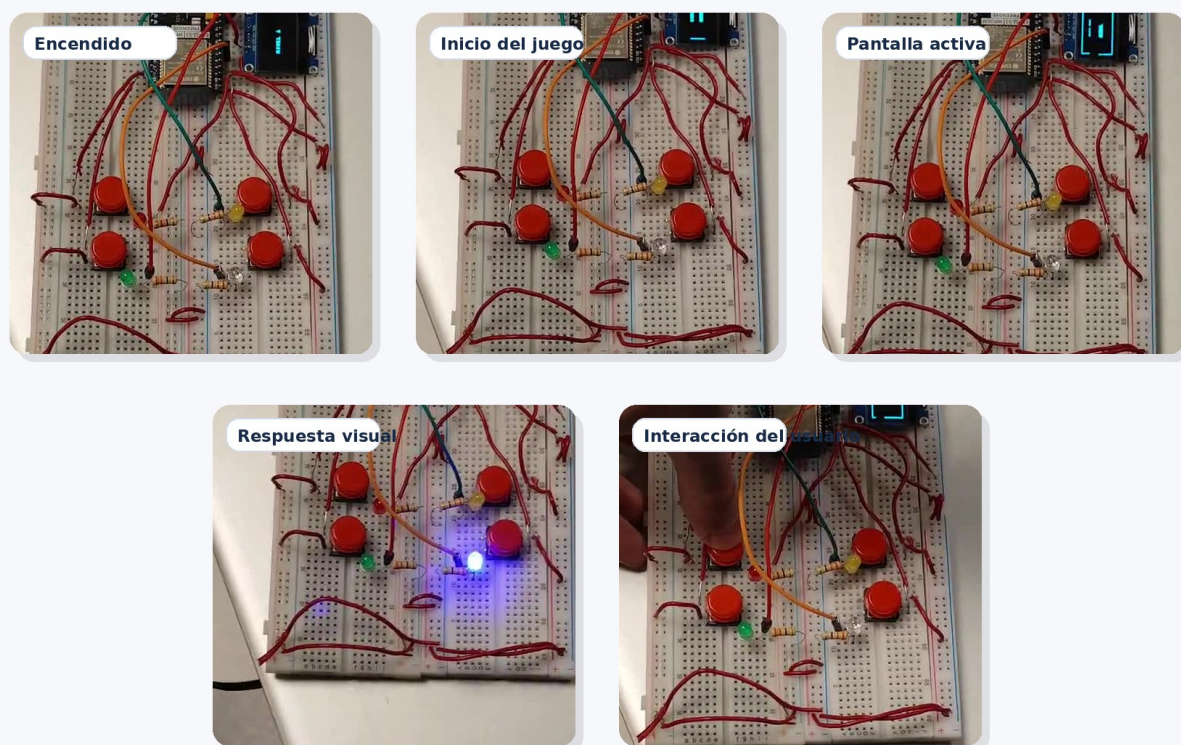


Figura 8. Capturas del video de funcionamiento entregado por el equipo.

El video resultó particularmente útil para la evaluación porque permite mostrar el comportamiento del sistema en secuencia: encendido, aparición de mensajes en pantalla, interacción del usuario y respuesta visual del prototipo. En una revisión exigente, la evidencia en movimiento ayuda a respaldar que no se trata de una sola fotografía tomada en un instante favorable.

Interpretación de la evidencia

De la evidencia fotográfica se desprende que el sistema logró integrar correctamente entradas y salidas alrededor del ESP32-S3. El display y el micrófono comparten la zona superior del montaje, mientras que los botones y LEDs se distribuyen en una cuadrícula que reproduce de forma intuitiva el patrón del juego. Ese arreglo facilitó las pruebas con usuarios porque la relación entre color, botón y respuesta visual pudo seguirse con rapidez.

También puede observarse que existieron cambios de cableado entre una toma y otra. Lejos de ser un defecto de la documentación, esto confirma que el montaje fue objeto de ajustes reales durante el proceso. En varias sesiones se reacomodaron jumpers, resistencias y trayectorias de conexión para reducir interferencias, simplificar el seguimiento del circuito y mejorar la estabilidad de la lectura.

Pruebas funcionales, incidencias y resultados

Para validar el prototipo no bastó con comprobar que encendiera. Se definió una batería de pruebas enfocada en arranque, percepción, despliegue de secuencia, captura de entrada, manejo de error y conservación del récord. La siguiente tabla resume los casos de prueba más importantes.

Caso	Procedimiento	Resultado esperado	Resultado observado	Conclusión
Arranque del sistema	Encender la placa y esperar inicialización	Pantalla con mensaje inicial y LEDs apagados	Se observó arranque limpio tras la rutina de limpieza	Cumplido
Inicio por botón	Presionar cualquier botón en estado IDLE	Cambio al modo de secuencia	El sistema respondió de manera consistente	Cumplido
Inicio por sonido	Generar aplauso o ruido por encima del umbral	Cambio al modo de secuencia	La respuesta fue correcta después de ajustar el umbral	Cumplido con ajuste
Visualización de secuencia	Observar ronda inicial y rondas posteriores	LEDs y pantalla deben seguir el mismo patrón	La sincronía general fue correcta	Cumplido
Captura de entrada	Repetir secuencia correcta	Avance de ronda y aumento de dificultad	La ronda se completó y se generó nuevo paso	Cumplido
Manejo de error	Ingresar un color incorrecto	Rutina de error y regreso a IDLE	Se ejecutó secuencia de error y reinicio controlado	Cumplido
Persistencia básica	Revisar datos en JSON tras juego	Actualización de récord y variables	La base de conocimientos conservó cambios esperados	Cumplido

Tabla 3. Casos de prueba funcional y resultados generales.

Las pruebas permitieron identificar incidencias que, de no haberse corregido, habrían puesto en riesgo la defensa del proyecto. Una de las más visibles fue el comportamiento errático al iniciar una nueva partida con datos residuales. Otra fue la inconsistencia ocasional del display al comunicarse por I2C. También se detectó la necesidad de un manejo más claro del rebote en botones para evitar dobles lecturas o avances falsos.

La respuesta a esas incidencias no consistió en “parchar” el sistema de forma improvisada. En cada caso se buscó localizar el origen del problema y dejar una corrección coherente con la arquitectura general. Ese criterio es importante porque muestra que el equipo no solamente logró que el juego corriera, sino que entendió por qué fallaba y cómo debía estabilizarse.

Principales incidencias detectadas y acciones correctivas

Incidencia	Causa probable	Acción correctiva	Impacto en la versión final
Inicio con comportamiento “loco” o arrastre de datos	Secuencia previa no limpiada al volver a IDLE	Limpieza explícita de secuencia y paso_actual al detectar nueva activación	Arranque más estable y repetible
LEDs que quedaban encendidos al reiniciar	Estado físico no limpiado al arrancar	Rutina de apagado total al inicio de main.py	Mejor presentación y menor confusión en laboratorio
Pantalla OLED con fallos esporádicos	Comunicación I2C sensible o energía inestable	Rutina de reconexión en caso de OSError	Mayor robustez del display
Lectura de sonido inestable	Umbral no ajustado al ambiente	Calibración de umbral y uso híbrido con botón	Inicio más controlado
Dificultad para explicar el código	Lógica concentrada o dispersa sin orden claro	Reorganización modular por carpetas y estados	Mejor defensa técnica y mantenimiento

Tabla 4. Incidencias relevantes observadas durante el desarrollo.

Esta tabla resume los problemas más importantes porque son precisamente los que suelen aparecer en una evaluación estricta. Poder señalar el problema, la causa probable y la solución aplicada transmite dominio del proceso y evita que la exposición quede reducida a una descripción superficial del prototipo.

Resultados observados y valoración del prototipo

En su estado final, el prototipo ejecuta de forma consistente el ciclo principal del juego. La secuencia se genera, la dificultad aumenta conforme avanzan las rondas y el usuario recibe retroalimentación visual suficiente para comprender el estado del sistema. La respuesta general es rápida, y la salida serial complementa la observación en pantalla al registrar eventos importantes durante la depuración.

La mayor fortaleza del proyecto no está en el número de componentes, sino en la forma en que quedaron articulados. El comportamiento observable coincide con la arquitectura lógica descrita en el código. Eso hace que el sistema sea defendible frente a preguntas técnicas sobre cómo decide, qué información conserva y por qué cambia de estado. Como punto mejorable, se reconoce que el montaje físico podría compactarse y presentarse en una versión con mejor acabado.

Análisis técnico del software implementado

El análisis de software se presenta por módulos para reflejar la organización real del proyecto. Cada bloque cumple una responsabilidad concreta y el conjunto funciona como una aplicación coordinada por estados. A continuación se describe la función de los archivos más importantes y la razón por la cual quedaron ubicados de esa manera.

main.py

Es el punto de entrada. Inicializa la base de conocimientos, LEDs, pantalla y sonido; limpia el hardware al arrancar y delega el control del juego a la máquina de estados. Su valor principal es que mantiene el ciclo principal lo más claro posible.

Observación técnica: main.py no se incluyó como relleno estructural. Su presencia dentro del repositorio corresponde a una necesidad concreta detectada durante la depuración. El diseño final muestra una transición desde un prototipo básico hacia una organización más limpia y mantenible.

core/kb_manager.py

Gestiona la base de conocimientos persistente. Carga el archivo JSON, permite consultar datos anidados, actualiza el estado actual y guarda cambios en memoria flash. Este archivo da coherencia al proyecto porque concentra conocimiento configurable y memoria de trabajo.

Observación técnica: core/kb_manager.py no se incluyó como relleno estructural. Su presencia dentro del repositorio corresponde a una necesidad concreta detectada durante la depuración. El diseño final muestra una transición desde un prototipo básico hacia una organización más limpia y mantenible.

core/state_machine.py

Declara los estados disponibles y administra las transiciones entre ellos. Evita que la lógica del flujo quede mezclada con detalles de hardware o validación.

Observación técnica: core/state_machine.py no se incluyó como relleno estructural. Su presencia dentro del repositorio corresponde a una necesidad concreta detectada durante la depuración. El diseño final muestra una transición desde un prototipo básico hacia una organización más limpia y mantenible.

core/inference.py

Implementa la regla central de inferencia: comparar la entrada del usuario contra la secuencia maestra almacenada. También se encarga de incrementar la puntuación y generar nuevos pasos con semilla aleatoria más robusta.

Observación técnica: core/inference.py no se incluyó como relleno estructural. Su presencia dentro del repositorio corresponde a una necesidad concreta detectada durante la depuración. El diseño final muestra una transición desde un prototipo básico hacia una organización más limpia y mantenible.

states/idle.py

Representa la fase de espera. Lee el micrófono y los botones para decidir si inicia una nueva partida. Además limpia rastros de la sesión anterior y prepara una breve animación de despertar.

Observación técnica: states/idle.py no se incluyó como relleno estructural. Su presencia dentro del repositorio corresponde a una necesidad concreta detectada durante la depuración. El diseño final muestra una transición desde un prototipo básico hacia una organización más limpia y mantenible.

states/sequence.py

Se encarga de construir o ampliar la secuencia y de reproducirla con tiempos dinámicos. A mayor nivel, menor duración de encendido y pausa, lo que incrementa la dificultad sin necesidad de reescribir la lógica.

Observación técnica: states/sequence.py no se incluyó como relleno estructural. Su presencia dentro del repositorio corresponde a una necesidad concreta detectada durante la depuración. El diseño final muestra una transición desde un prototipo básico hacia una organización más limpia y mantenible.

states/user_input.py

Lee los botones del usuario, aplica retroalimentación visual breve y delega la evaluación a InferenceEngine. Aquí también se ejecutan las transiciones a nueva ronda o a rutina de error.

Observación técnica: states/user_input.py no se incluyó como relleno estructural. Su presencia dentro del repositorio corresponde a una necesidad concreta detectada durante la depuración. El diseño final muestra una transición desde un prototipo básico hacia una organización más limpia y mantenible.

hardware/display.py

Controla la pantalla OLED. El detalle más relevante es la rutina de reinicialización cuando aparece un error de comunicación.

Observación técnica: hardware/display.py no se incluyó como relleno estructural. Su presencia dentro del repositorio corresponde a una necesidad concreta detectada durante la depuración. El diseño final muestra una transición desde un prototipo básico hacia una organización más limpia y mantenible.

hardware/leds.py

Agrupar la lógica de encendido, apagado general y secuencia de error. Su existencia evita repetir operaciones físicas en los estados.

Observación técnica: hardware/leds.py no se incluyó como relleno estructural. Su presencia dentro del repositorio corresponde a una necesidad concreta detectada durante la depuración. El diseño final muestra una transición desde un prototipo básico hacia una organización más limpia y mantenible.

hardware/sound.py

Preparó la salida de sonido y la integración con eventos del juego. Aunque el prototipo se apoyó también en disparadores seriales, el módulo documenta la intención de manejar audio como una responsabilidad independiente.

Observación técnica: hardware/sound.py no se incluyó como relleno estructural. Su presencia dentro del repositorio corresponde a una necesidad concreta detectada durante la depuración. El diseño final muestra una transición desde un prototipo básico hacia una organización más limpia y mantenible.

Base de conocimientos y parámetros persistentes

Uno de los aciertos del proyecto fue separar parámetros y estado actual dentro de bc_data.json. Allí se almacenan el mapa de hardware, los tiempos base de encendido y pausa, el factor de aceleración, el umbral mínimo, el umbral de ruido para el micrófono y variables de la memoria de trabajo como secuencia_maestra, paso_actual, puntuación y récord máximo.

Esta decisión permitió varias ventajas. Primero, facilitó la lectura del proyecto porque los valores importantes quedaron visibles en un solo archivo. Segundo, hizo posible ajustar el comportamiento sin modificar toda la aplicación. Tercero, reforzó la idea de que el sistema toma decisiones a partir de conocimiento consultable, no solamente a partir de constantes incrustadas dentro del código.

Máquina de estados y control del flujo

La máquina de estados es la pieza que le da disciplina al flujo general. En lugar de mezclar condiciones dispersas dentro de un while infinito, el sistema reconoce tres estados principales: IDLE, SEQUENCE y USER_INPUT. Cada uno tiene un propósito claro y una forma concreta de entrar en acción. Esa separación hizo más fácil razonar sobre el comportamiento del juego y encontrar errores de transición.

El beneficio adicional es comunicativo. Cuando un evaluador pregunta cómo sabe el sistema qué hacer en cada momento, la respuesta no queda en frases vagas. Se puede mostrar que existe una entidad central que conserva el estado actual y delega la ejecución al módulo correspondiente. Eso hace que la explicación sea más convincente.

Percepción, retroalimentación y depuración

Las entradas del sistema no se limitan a botones. El proyecto integra una lectura analógica de sonido, lo que añade un componente perceptivo que se alinea mejor con la materia. Aunque en la práctica el botón sigue siendo el disparador más estable, la activación híbrida amplía la experiencia del usuario y demuestra que el prototipo puede interpretar más de un tipo de evento de inicio.

La retroalimentación tampoco se dejó en un solo canal. Los LEDs, la pantalla y los mensajes seriales cumplen funciones complementarias. Los LEDs representan la secuencia y el feedback inmediato, la pantalla contextualiza el estado y el puerto serial sirve para depurar o sincronizar eventos de audio. Esta combinación resultó especialmente valiosa durante las sesiones finales, cuando fue necesario justificar con evidencia qué estaba ocurriendo dentro del sistema.

Consideraciones para repositorio, mantenimiento y crecimiento

Debido a que el proyecto también se preparó para respaldo en GitHub, se consideró importante que el repositorio no quedara como una carpeta caótica con archivos sueltos. La estructura final propone separar código, documentación, medios y recursos gráficos. Esta organización no solamente mejora la presentación; también reduce el riesgo de perder contexto entre una entrega y otra.

Desde el punto de vista de mantenimiento, el siguiente paso razonable sería normalizar todavía más el manejo de excepciones, encapsular mejor la parte de audio y definir una capa de configuración con valores por defecto y perfiles de entorno. También sería deseable encapsular físicamente el prototipo y elaborar un diagrama eléctrico más preciso para transición a PCB o a una placa perforada.

Recomendaciones concretas para una versión posterior:

- Diseñar un gabinete o base que mejore la manipulación del prototipo durante la exposición.
- Sustituir el protoboard por una solución más compacta y estable.
- Agregar buzzer dedicado o amplificación más controlada para el sonido.
- Registrar tiempos de respuesta y métricas de uso por partida.
- Añadir un modo de calibración para el sensor de audio según el ambiente.

Conclusiones y trabajo futuro

El proyecto cumplió con el propósito de la materia porque no se quedó en una demostración superficial. El equipo desarrolló un prototipo que percibe, conserva información, toma decisiones y responde de manera observable. La versión final ya puede explicarse con una arquitectura clara: base de conocimientos, inferencia, máquina de estados, percepción y actuadores. Ese orden conceptual es uno de los logros más importantes del trabajo.

También se concluye que la calidad de un proyecto no depende únicamente de que “funcione una vez”. En este caso, la parte más formativa estuvo en la depuración: limpiar estados residuales, corregir la inicialización del display, estabilizar lecturas y reorganizar módulos hasta que el comportamiento resultara consistente. Justamente por eso la documentación se amplió y se formalizó; había un proceso real que valía la pena dejar asentado.

Como trabajo futuro, el proyecto puede crecer hacia una versión más robusta y presentable. El paso inmediato sería mejorar el encapsulado y dejar una conexión física más limpia. Después podría añadirse almacenamiento de estadísticas por sesión, perfiles de dificultad o una interfaz externa para visualizar resultados. Incluso podría explorarse el reconocimiento más fino de patrones de sonido en lugar de un umbral simple.

En síntesis, el sistema “Simón Dice” desarrollado por el equipo constituye una base técnica legítima para seguir construyendo. La versión actual ya ofrece suficiente materia para defensa académica seria, y al mismo tiempo conserva el potencial de evolucionar hacia un prototipo con mejor terminación, mayor inteligencia perceptiva y más posibilidades de interacción.

Anexo A. Configuración base en JSON

```
{
  "hardware_map": {
    "1": {
      "color": "azul",
      "led": 7,
      "btn": 5,
      "note": 329
    },
    "0": {
      "color": "rojo",
      "led": 6,
      "btn": 1,
      "note": 261
    },
    "3": {
      "color": "amarillo",
      "led": 3,
      "btn": 4,
      "note": 523
    },
    "2": {
      "color": "verde",
      "led": 9,
      "btn": 2,
      "note": 392
    }
  },
  "estado_actual": {
    "record_maximo": 67,
    "paso_actual": 0,
    "puntuacion": 67,
    "secuencia_maestra": []
  },
  "reglas_inferencia": {
    "umbral_minimo_ms": 130,
    "ms_pausa_base": 200,
    "factor_aceleracion": 0.92,
    "ms_encendido_base": 400,
    "puntos_para_acelerar": 1
  },
  "percepcion_auditiva": {
    "valor_silencio": 3000,
    "modo_inicio": "hibrido",
    "mic_pin": 13,
    "umbral_ruido": 3400
  }
}
```

El archivo JSON concentra el mapa de hardware, las reglas de inferencia y la memoria de trabajo. Incluirlo en el reporte resulta útil porque permite relacionar la documentación con valores concretos del sistema.

Anexo B. main.py

Archivo incluido para revisión técnica. Se conserva su estructura original y se anexa como evidencia de implementación.

```
import time
from core.kb_manager import KBManager
from core.state_machine import SimonStateMachine
from hardware.leds import LEDController
from hardware.display import DisplayController
from hardware.sound import SoundController

def main():
    # 1. Inicialización de los componentes
    kb = KBManager()
    leds = LEDController(kb)
    display = DisplayController()

    # Si el sonido necesita un pin específico, se configura aquí
    sound = SoundController()

    # 2. LIMPIEZA DE HARDWARE (Para apagar LEDs amarillo/verde rebeldes)
    print("Limpiando hardware...")
    leds.todos_off()
    if display:
        display.clear()
        display.mostrar_resultado("SIMON DICE")

    # 3. Inicialización del Cerebro (Máquina de Estados)
    # Pasamos todos los controladores para que los estados puedan usarlos
    brain = SimonStateMachine(kb, leds, display, sound)

    print("Empieza el juego...")

    # 4. Bucle Principal
    while True:
        # La máquina de estados decide qué hacer según el estado actual
        brain.update()

        # Pequeña pausa para no saturar el procesador del ESP32
        time.sleep(0.01)

if __name__ == "__main__":
    main()
```

Anexo C. core/kb_manager.py

Archivo incluido para revisión técnica. Se conserva su estructura original y se anexa como evidencia de implementación.

```
import ujson
import os

class KBManager:
    _instance = None
    _data = None
    _path = "bc_data.json"

    def __new__(cls):
        """Implementación de Singleton para asegurar una única instancia."""
        if cls._instance is None:
            cls._instance = super(KBManager, cls).__new__(cls)
            cls._instance._load_kb()
        return cls._instance

    def _load_kb(self):
        """Carga la Base de Conocimientos desde el sistema de archivos."""
        try:
            with open(self._path, 'r') as f:
                self._data = ujson.load(f)
            print("KB: Base de conocimientos cargada con éxito.")
        except Exception as e:
            print(f"KB: Error al cargar JSON: {e}")
            # Datos por defecto en caso de error crítico
            self._data = {"estado_actual": {"secuencia_maestra": []}}

    def get(self, *keys):
        """
        Consulta inteligente: permite obtener datos anidados.
        Uso: kb.get('reglas_juego', 'ms_encendido')
        """
        temp = self._data
        for k in keys:
            if isinstance(temp, dict) and k in temp:
                temp = temp[k]
            else:
                return None
        return temp

    def update_state(self, key, value):
        """Actualiza la memoria de trabajo (RAM) y persiste en el JSON."""
        if "estado_actual" in self._data:
            self._data["estado_actual"][key] = value
            self.save()

    def save(self):
        """Persiste los cambios de la RAM al archivo flash del ESP32-S3."""
        try:
            with open(self._path, 'w') as f:
                ujson.dump(self._data, f)
        except Exception as e:
            print(f"KB: Error al guardar persistencia: {e}")

    def reset_sequence(self):
        """Regla de inferencia básica para reiniciar el juego."""
        self.update_state("secuencia_maestra", [])
```

```
self.update_state("paso_actual", 0)
```


Anexo D. core/state_machine.py

Archivo incluido para revisión técnica. Se conserva su estructura original y se anexa como evidencia de implementación.

```
from states.idle import IdleState
from states.sequence import SequenceState
from states.user_input import UserInputState

class SimonStateMachine:
    # CORRECCIÓN: El orden ahora es (kb, leds, display, sound) para coincidir con main.py
    def __init__(self, kb, leds, display, sound=None):
        self.kb = kb
        self.leds = leds
        self.display = display
        self.sound = sound

        self.states = {
            "IDLE": IdleState(self),
            "SEQUENCE": SequenceState(self),
            "USER_INPUT": UserInputState(self)
        }

        self.current_state = self.states["IDLE"]
        self.current_state.on_enter()

    def change_state(self, state_name):
        self.current_state = self.states[state_name]
        self.current_state.on_enter()

    def update(self):
        self.current_state.update()
```

Anexo E. core/inference.py

Archivo incluido para revisión técnica. Se conserva su estructura original y se anexa como evidencia de implementación.

```
import random
import os
import time

class InferenceEngine:
    def __init__(self, kb):
        self.kb = kb
        # --- FIX DE ENTROPÍA (REEMPLAZO DE machine.rng) ---
        # Generamos una semilla única usando bytes aleatorios del hardware
        try:
            semilla = int.from_bytes(os.urandom(4), 'little')
            random.seed(semilla)
        except:
            # Respaldo por si urandom fallara en algún boot
            random.seed(time.ticks_us())

    def validar_entrada(self, simbolo_recibido):
        """
        Aplica la regla de inferencia:
        SI el símbolo recibido coincide con el símbolo en la posición actual
        de la secuencia maestra, ENTONCES es un ACIERTO.
        """
        secuencia = self.kb.get("estado_actual", "secuencia_maestra")
        paso_actual = self.kb.get("estado_actual", "paso_actual")

        if not secuencia or paso_actual >= len(secuencia):
            return "ERROR_LOGICO"

        color_esperado = str(secuencia[paso_actual])

        if str(simbolo_recibido) == color_esperado:
            return self._procesar_acierto(paso_actual, len(secuencia))
        else:
            return "ERROR_USUARIO"

    def _procesar_acierto(self, paso_actual, total_secuencia):
        """
        Deduce si el acierto completa la secuencia o solo un paso.
        """
        nuevo_paso = paso_actual + 1
        self.kb.update_state("paso_actual", nuevo_paso)

        if nuevo_paso == total_secuencia:
            self._incrementar_puntuacion()
            return "SUCCESS_COMPLETE"

        return "SUCCESS_STEP"

    def _incrementar_puntuacion(self):
        """Actualiza los hechos en la BC al completar una ronda."""
        puntos = self.kb.get("estado_actual", "puntuacion") or 0
        nueva_puntuacion = puntos + 1
        self.kb.update_state("puntuacion", nueva_puntuacion)

        record = self.kb.get("estado_actual", "record_maximo") or 0
        if nueva_puntuacion > record:
```

```
self.kb.update_state("record_maximo", nueva_puntuacion)

def generar_nuevo_paso(self):
    """
    Regla: Añadir un nuevo símbolo aleatorio (0-3) a la secuencia.
    """
    secuencia = self.kb.get("estado_actual", "secuencia_maestra") or []

    # Agitamos la semilla antes de cada nuevo color usando entropía de hardware
    try:
        nueva_semilla = int.from_bytes(os.urandom(4), 'little')
        random.seed(nueva_semilla)
    except:
        random.seed(time.ticks_cpu())

    nuevo_simbolo = random.randint(0, 3)
    secuencia.append(nuevo_simbolo)

    self.kb.update_state("secuencia_maestra", secuencia)
    self.kb.update_state("paso_actual", 0)
    return secuencia
```

Anexo F. states/idle.py

Archivo incluido para revisión técnica. Se conserva su estructura original y se anexa como evidencia de implementación.

```
from machine import Pin, ADC
import time

class IdleState:
    def __init__(self, fsm):
        self.fsm = fsm
        self.kb = fsm.kb

    # --- PERCEPCIÓN: MICRÓFONO (GPIO 11) ---
    self.mic = ADC(Pin(13))
    self.mic.atten(ADC.ATTN_11DB)

    # Configuración de botones
    self.btn_rojo = Pin(1, Pin.IN, Pin.PULL_DOWN)
    self.btn_verde = Pin(2, Pin.IN, Pin.PULL_DOWN)
    self.btn_amarillo = Pin(4, Pin.IN, Pin.PULL_DOWN)
    self.btn_azul = Pin(5, Pin.IN, Pin.PULL_DOWN)

    def on_enter(self):
        print("Estado: IDLE. Esperando inicio (Botón o Aplauso)...")
        if self.fsm.leds:
            self.fsm.leds.todos_off()

        if self.fsm.display:
            self.fsm.display.mostrar_resultado("SIMON DICE")

    def update(self):
        # 1. LEER UMBRAL DEL JSON
        umbral = self.kb.get("percepcion_auditiva", "umbral_ruido") or 3400
        lectura_mic = self.mic.read()

        # 2. VERIFICAR ACTIVACIÓN
        btn_presionado = (self.btn_rojo.value() == 1 or self.btn_verde.value() == 1 or
                          self.btn_amarillo.value() == 1 or self.btn_azul.value() == 1)

        sonido_detectado = lectura_mic > umbral

        if btn_presionado or sonido_detectado:
            tipo = "SONIDO" if sonido_detectado else "BOTÓN"
            print(f"Percepción: ¡Inicio detectado por {tipo}! (Valor: {lectura_mic})")

            # --- LIMPIEZA DE MEMORIA (FIX PARA EL INICIO LOCO) ---
            # Borramos cualquier rastro de la partida anterior en la BC
            self.kb.update_state("secuencia_maestra", [])
            self.kb.update_state("paso_actual", 0)

            # Animación rápida de "Despertar"
            if self.fsm.leds: self.fsm.leds.todos_on()
            time.sleep(0.2)
            if self.fsm.leds: self.fsm.leds.todos_off()

            time.sleep(0.3)
            self.fsm.change_state("SEQUENCE")
```

Anexo G. states/sequence.py

Archivo incluido para revisión técnica. Se conserva su estructura original y se anexa como evidencia de implementación.

```
import time
import random

class SequenceState:
    def __init__(self, fsm):
        self.fsm = fsm
        self.kb = fsm.kb
        self.leds = fsm.leds
        self.display = fsm.display

    def on_enter(self):
        secuencia = self.kb.get("estado_actual", "secuencia_maestra")
        nivel = len(secuencia) if secuencia else 1

        if self.display:
            self.display.mostrar_resultado(f"NIVEL {nivel}")

        # ESPERA DINÁMICA: A más nivel, menos espera para empezar
        espera_inicial = max(0.2, 1.0 - (nivel * 0.1))
        time.sleep(espera_inicial)

        self.ejecutar_secuencia()

        if self.display:
            self.display.mostrar_resultado("TU TURNO")

        self.fsm.change_state("USER_INPUT")

    def ejecutar_secuencia(self):
        secuencia = self.kb.get("estado_actual", "secuencia_maestra")

        if not secuencia:
            secuencia = [random.choice(["0", "1", "2", "3"])]
            self.kb.update_state("secuencia_maestra", secuencia)

        # --- LÓGICA BASADA EN EL JSON ---
        nivel = len(secuencia)
        base_on = self.kb.get("reglas_inferencia", "ms_encendido_base") or 400
        base_off = self.kb.get("reglas_inferencia", "ms_pausa_base") or 200
        factor = self.kb.get("reglas_inferencia", "factor_aceleracion") or 0.92
        minimo = self.kb.get("reglas_inferencia", "umbral_minimo_ms") or 130

        # CÁLCULO DINÁMICO DE TIEMPOS
        # T = Base * (Factor ^ (Nivel - 1))
        duracion = max(minimo, int(base_on * (factor ** (nivel - 1))))
        pausa = max(int(minimo / 2), int(base_off * (factor ** (nivel - 1))))

        print(f"Log: Velocidad -> ON: {duracion}ms, OFF: {pausa}ms")

        nombres = {"0": "ROJO", "1": "AZUL", "2": "VERDE", "3": "AMARILLO"}

        # REPRODUCCIÓN DE LA SECUENCIA
        for simbolo in secuencia:
            simbolo_str = str(simbolo)
            nombre_color = nombres.get(simbolo_str, "???)
```

```
# --- INTEGRACIÓN DE SONIDO (LAPTOP) ---  
# Mandamos el ID al puerto serial para que el script de la PC lo cache  
print(f"[SND]:{simbolo_str}")  
  
if self.display:  
    self.display.mostrar_color(nombre_color)  
  
self.leds.encender(simbolo_str, duracion)  
  
# Tiempo de espera entre destellos  
time.sleep_ms(pausa)
```

Anexo H. states/user_input.py

Archivo incluido para revisión técnica. Se conserva su estructura original y se anexa como evidencia de implementación.

```
import time
from machine import Pin
from core.inference import InferenceEngine

class UserInputState:
    def __init__(self, fsm):
        self.fsm = fsm
        self.kb = fsm.kb
        self.leds = fsm.leds
        self.display = fsm.display
        self.inference = InferenceEngine(self.kb)

        # CONFIGURACIÓN FÍSICA DIRECTA (Pines ISC-UNITEC validados)
        self.btn_rojo = Pin(1, Pin.IN, Pin.PULL_DOWN) # ID 0
        self.btn_azul = Pin(5, Pin.IN, Pin.PULL_DOWN) # ID 1
        self.btn_verde = Pin(2, Pin.IN, Pin.PULL_DOWN) # ID 2
        self.btn_amarillo = Pin(4, Pin.IN, Pin.PULL_DOWN) # ID 3

    def on_enter(self):
        print("Estado: USER_INPUT. Esperando tu respuesta...")
        if self.display:
            self.display.mostrar_resultado("TU TURNO")

        self.kb.update_state("paso_actual", 0)

    def update(self):
        entrada = None

        # 1. Escaneo de pines
        if self.btn_rojo.value() == 1: entrada = "0"
        elif self.btn_azul.value() == 1: entrada = "1"
        elif self.btn_verde.value() == 1: entrada = "2"
        elif self.btn_amarillo.value() == 1: entrada = "3"

        if entrada is not None:
            # --- LECTURA DE SENSIBILIDAD DESDE LA BC ---
            t_feedback = self.kb.get("reglas_inferencia", "feedback_usuario_ms") or 80
            t_debounce = self.kb.get("reglas_inferencia", "debounce_ms") or 60

            nombres = {"0": "ROJO", "1": "AZUL", "2": "VERDE", "3": "AMARILLO"}

            # --- DISPARADOR DE SONIDO (LAPTOP) ---
            print(f"[SND]:{entrada}")

            if self.display:
                self.display.mostrar_color(nombres.get(entrada))

            # Feedback físico
            self.leds.encender(entrada, t_feedback)

            # Espera mínima de rebote definida en JSON
            time.sleep_ms(t_debounce)

            self.procesar_logica(entrada)

    def procesar_logica(self, simbolo):
```

```

resultado = self.inference.validar_entrada(simbolo)

if resultado == "SUCCESS_STEP":
    print(f"¡Bien! ID {simbolo} correcto.")

elif resultado == "SUCCESS_COMPLETE":
    # --- DISPARADOR DE SONIDO: ÉXITO ---
    print("[SND]:SUCCESS")

    print("¡Ronda superada!")
    time.sleep(0.4)
    self.inference.generar_nuevo_paso()
    self.fsm.change_state("SEQUENCE")

elif resultado == "ERROR_USUARIO":
    # --- DISPARADOR DE SONIDO: ERROR ---
    print("[SND]:ERROR")

    print("¡Error! Secuencia incorrecta.")
    self.leds.secuencia_error()

    secuencia = self.kb.get("estado_actual", "secuencia_maestra")
    puntos = len(secuencia) if secuencia else 0

    record = self.kb.get("estado_actual", "record_maximo") or 0
    if puntos > record:
        print(f"¡NUEVO RÉCORD: {puntos}!")
        self.kb.update_state("record_maximo", puntos)

    if self.display:
        self.display.mostrar_resultado(f"FIN. PTS: {puntos}")

    time.sleep(1.5)
    self.kb.update_state("secuencia_maestra", [])
    self.fsm.change_state("IDLE")

```


Anexo I. hardware/display.py

Archivo incluido para revisión técnica. Se conserva su estructura original y se anexa como evidencia de implementación.

```
import time
from machine import Pin, SoftI2C
import ssd1306

class DisplayController:
    def __init__(self, width=128, height=64):
        self.width = width
        self.height = height
        self.iniciar_i2c()
        print("Display: Modo visual activo.")

    def iniciar_i2c(self):
        """Crea la conexión básica con la pantalla."""
        try:
            self.i2c = SoftI2C(sda=Pin(18, Pin.IN, Pin.PULL_UP), scl=Pin(17, Pin.IN, Pin.PULL_UP), freq=10000)
            self.oled = ssd1306.SSD1306_I2C(self.width, self.height, self.i2c)
            # Damos un pequeño respiro a la energía
            time.sleep(0.1)
        except Exception as e:
            self.oled = None

    def mostrar_resultado(self, mensaje):
        """Muestra texto. Si la energía falla, atrapa el error y se reinicia sola."""
        if not self.oled: return
        try:
            self.oled.fill(0)
            self.oled.text(mensaje, 30, 30)
            self.oled.show()
        except OSError:
            # Mecanismo de rescate básico: reconectamos y lo intentamos de nuevo
            self.iniciar_i2c()
            if self.oled:
                self.oled.fill(0)
                self.oled.text(mensaje, 30, 30)
                self.oled.show()

    def mostrar_color(self, nombre):
        if not self.oled: return
        try:
            self.oled.fill(0)
            self.oled.rect(0, 0, 128, 64, 1)
            self.oled.text("COLOR:", 40, 15)
            self.oled.text(nombre.upper(), 40, 35)
            self.oled.show()
        except OSError:
            # Mecanismo de rescate básico
            self.iniciar_i2c()
            if self.oled:
                self.oled.fill(0)
                self.oled.rect(0, 0, 128, 64, 1)
                self.oled.text("COLOR:", 40, 15)
                self.oled.text(nombre.upper(), 40, 35)
                self.oled.show()

    def clear(self):
        if not self.oled: return
```

```
try:
    self.oled.fill(0)
    self.oled.show()
except OSError:
    self.iniciar_i2c()
```

Anexo J. hardware/leds.py

Archivo incluido para revisión técnica. Se conserva su estructura original y se anexa como evidencia de implementación.

```
from machine import Pin
import time

class LEDController:
    def __init__(self, kb):
        self.fsm_kb = kb # Guardamos la referencia a la BC
        self.leds = {}

        # En lugar de get_all, buscamos los IDs directamente 0, 1, 2, 3
        # Esto es más robusto y evita el error de atributo
        for i in range(4):
            id_str = str(i)
            # Buscamos el pin en hardware_map -> id -> led
            pin_num = self.fsm_kb.get("hardware_map", id_str).get("led")
            if pin_num:
                self.leds[id_str] = Pin(pin_num, Pin.OUT)
                self.leds[id_str].value(0)
                print(f"LED {id_str} configurado en pin {pin_num}")

    def encender(self, id_color, ms):
        id_str = str(id_color)
        if id_str in self.leds:
            self.leds[id_str].value(1)
            time.sleep_ms(ms)
            self.leds[id_str].value(0)

    def todos_off(self):
        for led in self.leds.values():
            led.value(0)

    def todos_on(self):
        for led in self.leds.values():
            led.value(1)

    def secuencia_error(self):
        # El parpadeo de "Game Over"
        for _ in range(3):
            self.todos_on()
            time.sleep_ms(150)
            self.todos_off()
            time.sleep_ms(150)
```

Anexo K. hardware/sound.py

Archivo incluido para revisión técnica. Se conserva su estructura original y se anexa como evidencia de implementación.

```
from machine import I2S, Pin
import math
import time
import struct

class SoundController:
    def __init__(self):
        self.audio_out = I2S(1,
                             sck=Pin(18), ws=Pin(8), sd=Pin(17),
                             mode=I2S.TX,
                             bits=16,
                             format=I2S.MONO,
                             rate=22050,
                             ibuf=2048)

        self.tonos = {
            0: 261,
            1: 329,
            2: 392,
            3: 523
        }

    def _generate_sine_wave(self, freq, duration_ms, volume=3000):
        """Genera un buffer de audio con una onda senoidal."""
        sample_rate = 22050
        num_samples = int(sample_rate * duration_ms / 1000)
        format_str = "<" + "h" * num_samples
        samples = []
        for i in range(num_samples):
            val = int(volume * math.sin(2 * math.pi * freq * i / sample_rate))
            samples.append(val)
        return struct.pack(format_str, *samples)

    def play_color_tone(self, id_color, duration=400):
        """Reproduce el tono específico de un color."""
        if id_color in self.tonos:
            freq = self.tonos[id_color]
            wave = self._generate_sine_wave(freq, duration)
            self.audio_out.write(wave)

    def play_success_tone(self):
        """Sonido rápido de acierto."""
        wave = self._generate_sine_wave(880, 100, volume=2000)
        self.audio_out.write(wave)

    def play_victory_melody(self):
        """Pequeña melodía al completar la secuencia."""
        for f in [523, 659, 783, 1046]:
            self.audio_out.write(self._generate_sine_wave(f, 150))
            time.sleep_ms(50)

    def play_error_tone(self):
        """Sonido grave de error."""
        wave = self._generate_sine_wave(100, 500, volume=5000)
        self.audio_out.write(wave)
```

```
def deinit(self):  
    """Libera el bus I2S."""  
    self.audio_out.deinit()
```

Anexo L. perception/button_adapter.py

Archivo incluido para revisión técnica. Se conserva su estructura original y se anexa como evidencia de implementación.

```
from machine import Pin
import time

class ButtonAdapter:
    def __init__(self, kb):
        self.kb = kb
        self.buttons = {}

        mapa = self.kb.get("hardware_map")

        if mapa is None:
            print("Percepción: Error - No se encontró 'hardware_map' en la BC")
            return

        for key, data in mapa.items():
            try:
                pin_num = data["btn"]
                # CAMBIO 1: Usamos PULL_DOWN para tu conexión a 3.3V
                self.buttons[int(key)] = Pin(pin_num, Pin.IN, Pin.PULL_DOWN)
                print(f"Percepción: Botón {data['color']} (ID {key}) configurado en pin {pin_num}")
            except Exception as e:
                print(f"Percepción: Error en botón {key}: {e}")

        self.debounce_time = 200

    def get_input(self):
        """
        Escanea los botones y devuelve el ID (0-3) si uno es presionado.
        """
        for id_color, pin in self.buttons.items():
            # CAMBIO 2: Detectamos el botón con 1 (High) porque mandas 3.3V
            if pin.value() == 1:
                time.sleep_ms(self.debounce_time)
                print(f"Percepción: Botón detectado -> ID {id_color}")
                # Devolvemos el ID numérico (0, 1, 2, 3)
                return id_color

        return None

    def wait_for_any_button(self):
        while True:
            res = self.get_input()
            if res is not None:
                return res
            time.sleep_ms(50)
```

Anexo M. perception/voice_adapter.py

Archivo incluido para revisión técnica. Se conserva su estructura original y se anexa como evidencia de implementación.

```
from machine import I2S, Pin
import time
import math

class VoiceAdapter:
    def __init__(self):
        self.audio_in = I2S(0,
                             sck=Pin(12), ws=Pin(11), sd=Pin(10),
                             mode=I2S.RX,
                             bits=16,
                             format=I2S.MONO,
                             rate=16000,
                             ibuf=16000)

        self.buffer = bytearray(1024)
        self.umbral_voz = 500

    def listen_for_color(self):
        """
        Escucha el ambiente. Si detecta sonido, intenta machear con un color.
        Retorna el ID (0-3) o None si no hay entrada clara.
        """
        num_read = self.audio_in.readinto(self.buffer)
        if num_read == 0:
            return None

        samples = self._bytes_to_samples(self.buffer)

        energy = self._calculate_energy(samples)

        if energy > self.umbral_voz:
            print(f'Percepción: Voz detectada (E:{energy})')
            return self._classify_voice(samples)

        return None

    def _bytes_to_samples(self, buf):
        import struct
        count = len(buf) // 2
        return struct.unpack(str(count) + 'h', buf)

    def _calculate_energy(self, samples):
        return math.sqrt(sum(s*s for s in samples) / len(samples))

    def _classify_voice(self, samples):
        """
        MOTOR DE INFERENCIA ACÚSTICO:
        Distingue colores por 'Firma de Frecuencia' o 'Duración'.
        """
        crossings = 0
        for i in range(len(samples)-1):
            if (samples[i] > 0 and samples[i+1] < 0) or (samples[i] < 0 and samples[i+1] > 0):
                crossings += 1

        if crossings > 150: return 3
        if crossings < 80: return 0
```

return None

Complemento de requerimientos y trazabilidad

Con base en la retroalimentación recibida por el equipo, se reforzó la parte de ingeniería de requerimientos para que el expediente mostrara no sólo la evidencia de construcción, sino también la lógica de trazabilidad entre lo pedido, lo implementado y lo validado en laboratorio.

La intención de esta sección es dejar un bloque más técnico y ordenado. Por eso se consolidaron requisitos funcionales y no funcionales, además de una matriz breve de correspondencia con módulos reales del sistema.

17.1 Requerimientos funcionales consolidados

ID	Descripción resumida	Prioridad	Verificación
RF-01	Cargar base de conocimientos y parámetros al encender.	Alta	Arranque y lectura de JSON
RF-02	Esperar activación por botón o sonido válido.	Alta	Prueba de inicio
RF-03	Limpiar secuencia previa antes de una nueva partida.	Alta	Reinicio controlado
RF-04	Generar y ampliar secuencia maestra por nivel.	Alta	Rondas sucesivas
RF-05	Mostrar secuencia con LEDs, OLED y apoyo serial.	Alta	Observación directa
RF-06	Capturar cuatro entradas físicas asociadas a color.	Alta	Prueba por botón
RF-07	Validar cada entrada contra la secuencia esperada.	Alta	Aciertos y errores
RF-08	Distinguir acierto parcial, ronda completa y fallo.	Alta	Revisión de inferencia
RF-09	Actualizar puntuación y récord cuando corresponda.	Media	Ensayo de marcador
RF-10	Ejecutar rutina de error y regresar a IDLE.	Alta	Fallo inducido
RF-11	Conservar parámetros ajustables en JSON.	Media	Edición y recarga
RF-12	Permitir depuración serial de eventos y estados.	Media	Monitoreo por IDE

Tabla 17. Requerimientos funcionales consolidados.

17.2 Requerimientos no funcionales y trazabilidad

ID	Requisito no funcional	Criterio de evaluación
RNF-01	Respuesta continua y clara para el usuario.	Experiencia de uso
RNF-02	Código modular por responsabilidades.	Estructura del repositorio
RNF-03	Parámetros relevantes accesibles sin reescribir todo.	Archivo JSON
RNF-04	Mensajes legibles en pantalla OLED.	Prueba visual
RNF-05	Debounce suficiente para evitar dobles	Prueba repetitiva

	lecturas.	
RNF-06	Estabilidad en reinicios y nuevas partidas.	Ensayo continuo
RNF-07	Facilidad de mantenimiento y mejora futura.	Revisión técnica
RNF-08	Documentación capaz de explicar decisiones y correcciones.	Evaluación documental

Tabla 18. Requerimientos no funcionales considerados en la revisión final.

Para cerrar el apartado, se resume la correspondencia más importante entre requisitos y módulos implementados. Se evitaron matrices excesivas; la prioridad fue mostrar relaciones útiles y verificables.

Requisito	Módulo	Implementación	Evidencia
RF-02	states/idle.py	Lectura de micrófono y botones	Inicio por sonido o pulsación
RF-04	states/sequence.py	Generación y crecimiento de secuencia	Rondas consecutivas
RF-06	states/user_input.py	Mapeo físico de cuatro botones	Pruebas por color
RF-07	core/inference.py	Comparación con secuencia maestra	Aciertos / errores
RF-10	user_input.py + leds.py	Rutina de fallo y reinicio	Video de funcionamiento
RNF-02	Estructura por carpetas	Separación de responsabilidades	Repositorio organizado

Tabla 19. Matriz compacta de trazabilidad.

Casos de uso y escenarios operativos

La revisión también pidió formalizar la interacción del sistema mediante casos de uso. Esto permite explicar el prototipo desde la operación y no sólo desde el montaje electrónico. Los casos seleccionados corresponden a lo que realmente hace la versión final del proyecto.

Se identificaron dos actores: el jugador, que interactúa con la partida; y el equipo técnico, que interviene en la calibración de parámetros y en el mantenimiento de la base de conocimientos.

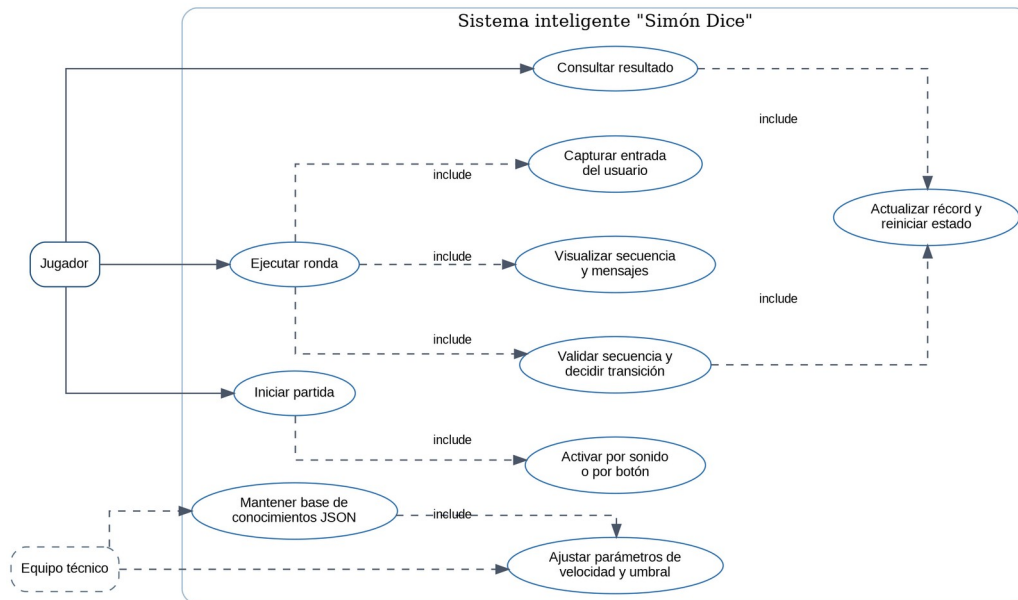


Figura 20. Diagrama general de casos de uso del sistema.

ID	Caso de uso	Actor	Resultado esperado
UC-01	Iniciar partida	Jugador	Se limpia el estado previo y se prepara la secuencia.
UC-02	Ejecutar ronda	Jugador / Sistema	Se reproduce la secuencia y se cede turno al usuario.
UC-03	Registrar entrada	Jugador	Se detecta el color presionado y se da retroalimentación.
UC-04	Validar secuencia	Sistema	Se decide continuidad, éxito o error.
UC-05	Consultar resultado	Jugador	Se muestra fin de ronda, puntaje o reinicio.
UC-06	Ajustar parámetros	Equipo técnico	Se recalibran umbrales y tiempos de juego.

Tabla 20. Resumen de casos de uso principales.

18.1 Escenarios resumidos de operación

Caso	Flujo resumido	Salida / postcondición
UC-01	El sistema detecta aplauso o botón, limpia variables temporales, ejecuta una animación breve y cambia al estado de secuencia.	La partida queda lista para comenzar sin arrastre de datos anteriores.

UC-02	La lógica calcula velocidad según nivel, reproduce la secuencia con LEDs y OLED y anuncia el turno del jugador.	El usuario recibe la referencia correcta antes de responder.
UC-04	El motor compara cada color recibido con la posición esperada, actualiza el paso actual y decide si la ronda continúa o se reinicia.	Se conserva coherencia entre secuencia, puntuación y transición de estados.
UC-06	El equipo revisa o modifica umbral de ruido, debounce y factor de aceleración dentro del JSON; después reinicia para validar.	La base de conocimientos queda ajustada a las condiciones reales del laboratorio.

Tabla 21. Escenarios operativos resumidos por caso de uso.

Diagramas de actividades por caso de uso

Para cerrar la mejora documental se añadieron diagramas de actividad. Con ellos se hace visible el orden de decisiones del prototipo: primero la detección de inicio, luego la reproducción de la secuencia, después la validación de la entrada y, por último, la ruta de error o de avance de ronda.

19.1 Flujo principal del sistema

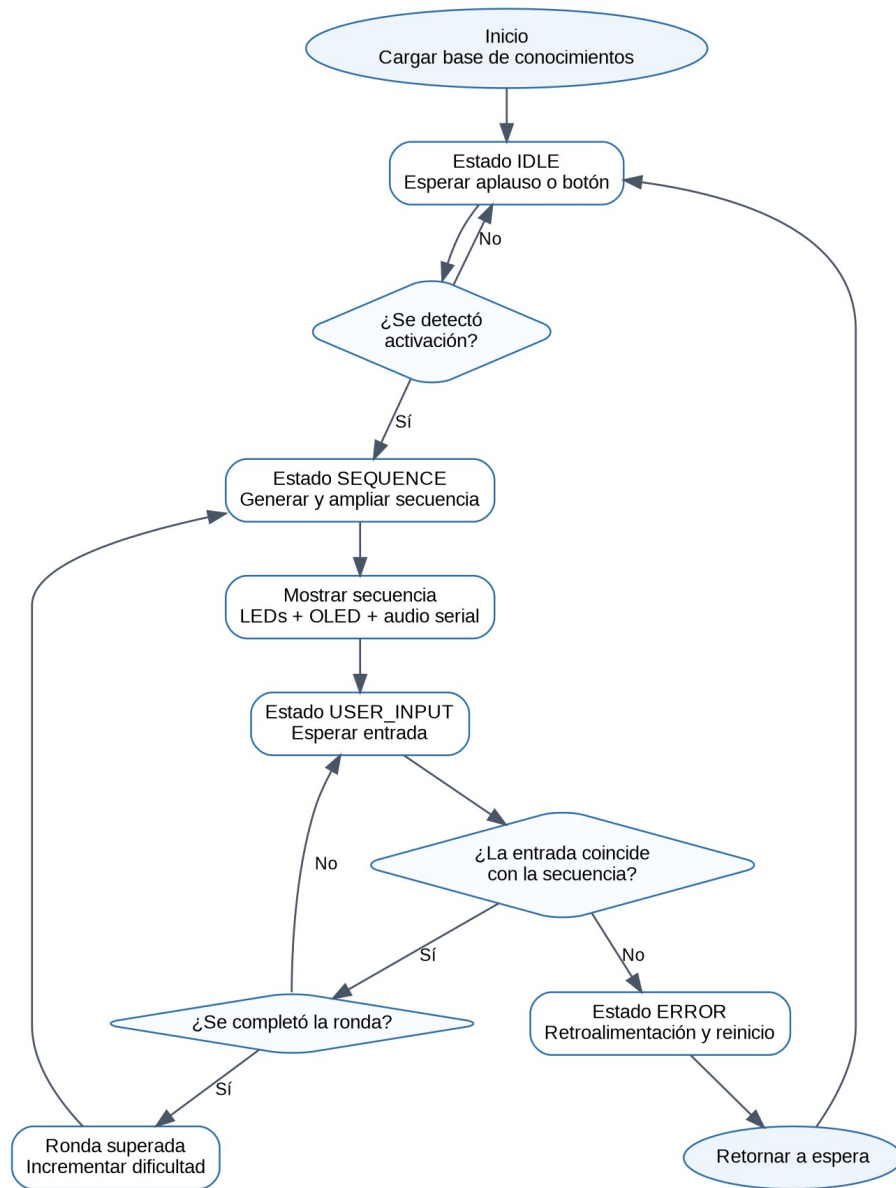
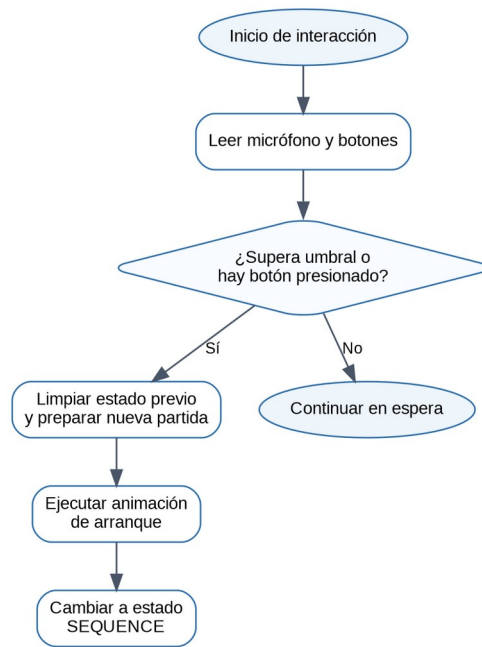
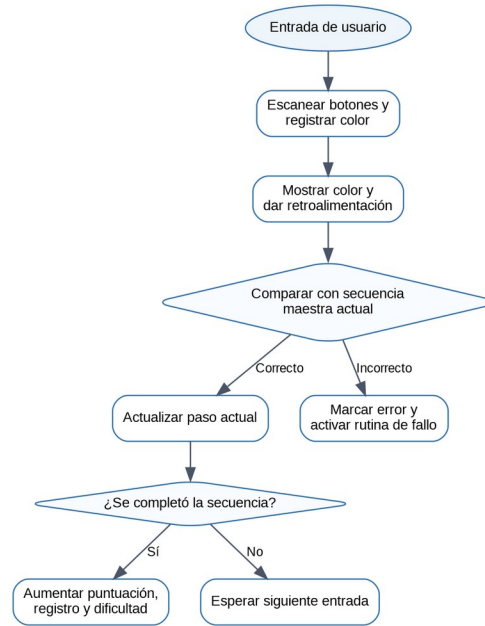


Figura 21. Flujo principal del sistema a partir del modelo de estados.

19.2 Actividades complementarias de casos de uso



UC-01. Inicio de partida



UC-04. Validación de secuencia

